

Conditional Statements in Java

Conditional statements control the flow of a program based on certain conditions. They allow the execution of different code blocks based on whether a condition evaluates to true or false.

Types of Conditional Statements

1. if Statement

Executes a block of code if the condition is true.

Syntax:

```
if (condition) {  
    // Code to execute if condition is true  
}
```

Example:

```
int age = 20;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. if-else Statement

Executes one block of code if the condition is true, and another if it is false.

Syntax:

```
if (condition) {  
    // Code to execute if condition is true  
} else {  
    // Code to execute if condition is false  
}
```

Example:

```
int number = 10;  
if (number % 2 == 0) {  
    System.out.println("The number is even.");  
} else {  
    System.out.println("The number is odd.");  
}
```

3. if-else-if Ladder

Tests multiple conditions sequentially. If one condition is true, its block executes, and the rest are ignored.

Syntax:

```
if (condition1) {  
    // Code for condition1  
} else if (condition2) {  
    // Code for condition2  
} else {  
    // Code if none of the conditions are true  
}
```

Example:

```
int marks = 85;  
if (marks >= 90) {  
    System.out.println("Grade: A");  
} else if (marks >= 75) {  
    System.out.println("Grade: B");  
} else if (marks >= 50) {  
    System.out.println("Grade: C");  
} else {  
    System.out.println("Grade: F");  
}
```

4. Nested if

An if statement inside another if statement.

Syntax:

```
if (condition1) {  
    if (condition2) {  
        // Code if both conditions are true  
    }  
}
```

Example:

```
int age = 25;

boolean hasVoterID = true;

if (age >= 18) {
    if (hasVoterID) {
        System.out.println("You can vote.");
    } else {
        System.out.println("You need a voter ID to vote.");
    }
} else {
    System.out.println("You are not eligible to vote.");
}
```

Switch Statement

The switch statement executes one block of code based on the value of a variable or expression. It is used when multiple values need to be checked against a variable.

Syntax:

```
switch (expression) {
    case value1:
        // Code for value1
        break;
    case value2:
        // Code for value2
        break;
    default:
        // Code if none of the cases match
}
```

Example:

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
```

case 2:

```
System.out.println("Tuesday");
```

```
break;
```

case 3:

```
System.out.println("Wednesday");
```

```
break;
```

default:

```
System.out.println("Invalid day");
```

```
}
```

Introduction to Loops

Loops allow the execution of a block of code multiple times. They are used to repeat tasks efficiently.

Types of Loops

1. for Loop

Repeats a block of code a specified number of times.

```
for (initialization; condition; update) {
```

```
// Code to execute
```

```
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
```

```
System.out.println("Count: " + i);
```

```
}
```

2. while Loop

Repeats a block of code as long as the condition is true.

Syntax:

```
while (condition) {
```

```
// Code to execute
```

```
}
```

Example:

```
int i = 1;
```

```
while (i <= 5) {
```

```
System.out.println("Count: " + i);  
  
i++;  
  
}
```

3. do-while Loop

Executes the code at least once and then repeats as long as the condition is true.

Syntax:

```
do {  
  
    // Code to execute  
  
} while (condition);
```

Example:

```
int i = 1;  
  
do {  
  
    System.out.println("Count: " + i);  
  
    i++;  
  
} while (i <= 5);
```

4. Enhanced for Loop (For-Each)

Used for iterating over arrays or collections.

Syntax:

```
for (type element : array) {  
  
    // Code to execute  
  
}
```

Example:

```
int[] numbers = {1, 2, 3, 4, 5};  
  
for (int num : numbers) {  
  
    System.out.println("Number: " + num);  
  
}
```